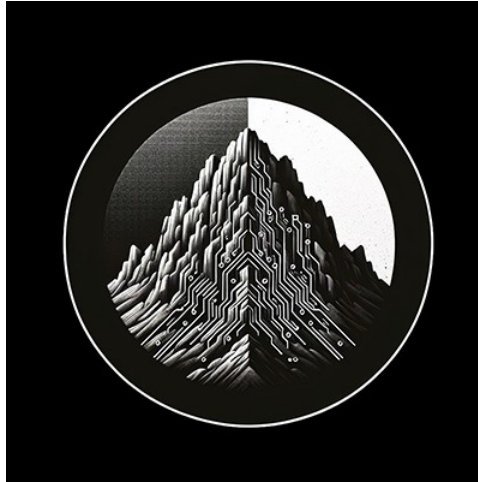




RTL Design Sherpa

APB SMBus Micro-Architecture Specification 1.0

January 4, 2026

Table of Contents

1 Smbus Mas Index.....	7
2 APB SMBus - Overview.....	7
2.1 Introduction.....	7
2.2 Key Features.....	7
2.3 Applications.....	8
2.4 Block Diagram.....	8
2.4.1 Figure 1.1: SMBus Block Diagram.....	8
2.5 Timing Diagrams.....	8
2.5.1 Waveform 1.1: Byte Write (Start + Address).....	8
2.5.2 Waveform 1.2: Byte Read.....	9
2.5.3 Waveform 1.3: Clock Stretching.....	9
2.5.4 Waveform 1.4: Multi-Master Arbitration.....	9
2.5.5 Waveform 1.5: Packet Error Check (PEC).....	10
2.6 Register Summary.....	10
3 APB SMBus - Register Map.....	11
3.1 Register Summary.....	11
3.2 SMBUS_CONTROL (0x00).....	13
3.3 SMBUS_STATUS (0x04).....	13
3.4 SMBUS_COMMAND (0x08).....	14
3.4.1 Transaction Types.....	14
3.5 SMBUS_SLAVE_ADDR (0x0C).....	15
3.6 SMBUS_DATA (0x10).....	15

3.7 SMBUS_TX_FIFO (0x14).....	15
3.8 SMBUS_RX_FIFO (0x18).....	16
3.9 SMBUS_FIFO_STATUS (0x1C).....	16
3.10 SMBUS_CLK_DIV (0x20).....	16
3.11 SMBUS_TIMEOUT (0x24).....	17
3.12 SMBUS_OWN_ADDR (0x28).....	17
3.13 SMBUS_INT_ENABLE (0x2C).....	17
3.14 SMBUS_INT_STATUS (0x30).....	18
3.15 SMBUS_PEC (0x34).....	19
3.16 SMBUS_BLOCK_COUNT (0x38).....	19
3.17 Implementation Limitations.....	19

List of Figures

Figure 1.1: SMBus Block Diagram.....	8
--------------------------------------	---

List of Tables

No tables in this document.

List of Waveforms

Waveform 1.1: Byte Write (Start + Address).....	8
Waveform 1.2: Byte Read.....	9
Waveform 1.3: Clock Stretching.....	9
Waveform 1.4: Multi-Master Arbitration.....	9
Waveform 1.5: Packet Error Check (PEC).....	10

1 Smbus Mas Index

Generated: 2026-08-14

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

2 APB SMBus - Overview

2.1 Introduction

The APB SMBus controller provides System Management Bus communication with APB interface. It supports host controller functionality for accessing SMBus devices.

2.2 Key Features

- SMBus 2.0 compatible
- Host controller mode
- Quick Command, Send/Receive Byte
- Read/Write Byte/Word
- Block Read/Write (up to 32 bytes)
- PEC (Packet Error Checking) support
- Programmable clock divider
- Interrupt-driven operation
- Timeout detection

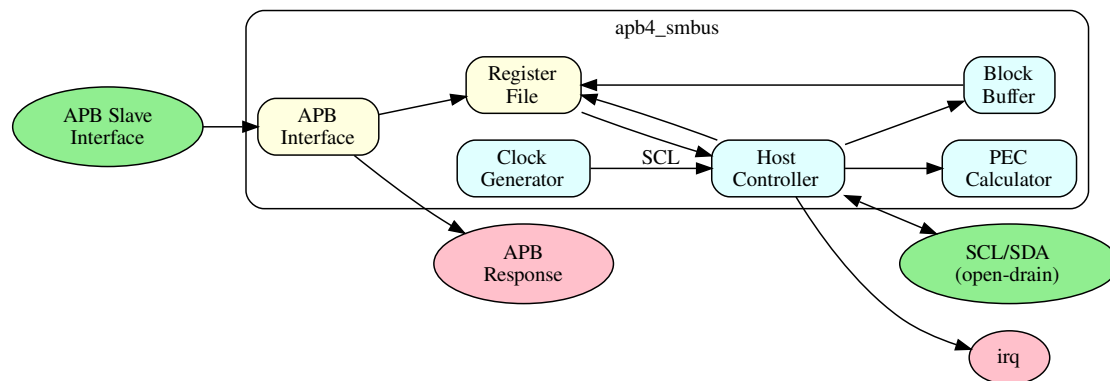
Implementation status: Several protocol features described in this chapter are not fully realized in the current RTL. Timeout detection, PEC generation/verification, multi-master arbitration, clock stretching, and slave mode are present in the register interface but not functional in `smbus_core` today. The timing diagrams below (Waveforms 1.3-1.5 in particular) illustrate SMBus protocol intent, not current hardware behavior. See the Implementation Limitations section of [Chapter 5: Register Map](#) and `rtl/smbus/IMPLEMENTATION_STATUS.md` for the tracked status.

2.3 Applications

- Temperature monitoring
- Voltage monitoring
- Fan control
- EEPROM access
- Power management
- System health monitoring

2.4 Block Diagram

2.4.1 Figure 1.1: SMBus Block Diagram

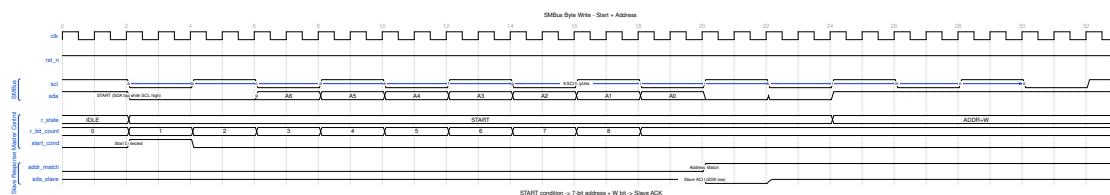


SMBus Block Diagram

2.5 Timing Diagrams

2.5.1 Waveform 1.1: Byte Write (Start + Address)

Shows the START condition and 7-bit address transmission.

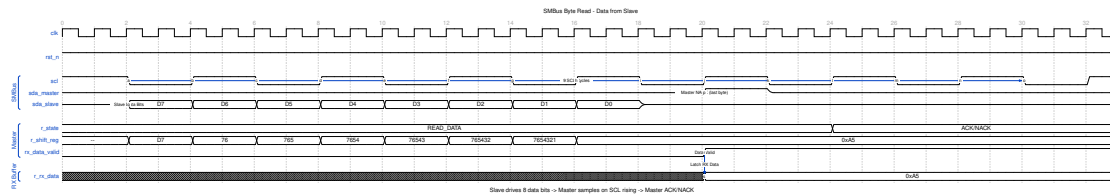


SMBus Byte Write

START condition is SDA falling while SCL is high. The 7-bit slave address plus R/W bit is clocked out, followed by slave ACK (SDA low during 9th clock).

2.5.2 Waveform 1.2: Byte Read

Shows slave-to-master data transfer.

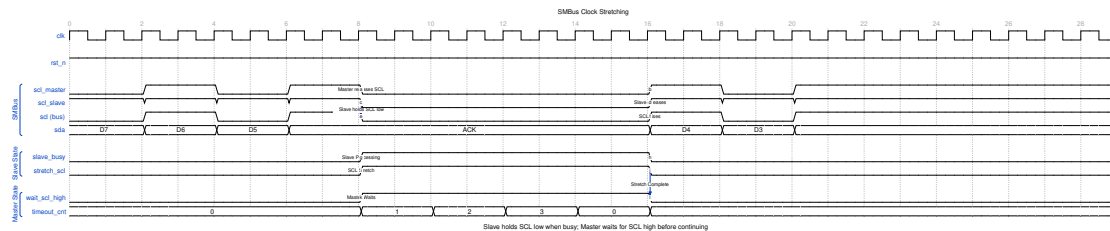


SMBus Byte Read

Slave drives 8 data bits while master clocks SCL. Master samples each bit on SCL rising edge, then provides ACK (more data) or NACK (last byte).

2.5.3 Waveform 1.3: Clock Stretching

Slave flow control by holding SCL low.

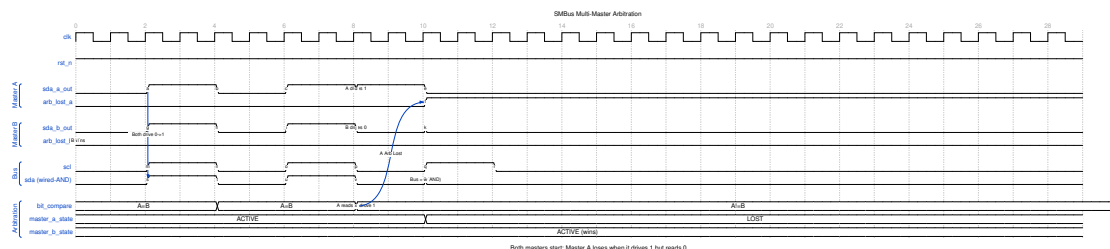


SMBus Clock Stretch

When the slave needs processing time, it holds SCL low after the master releases it. Master waits for SCL to rise before continuing. This provides backpressure without data loss.

2.5.4 Waveform 1.4: Multi-Master Arbitration

Collision detection when multiple masters start simultaneously.

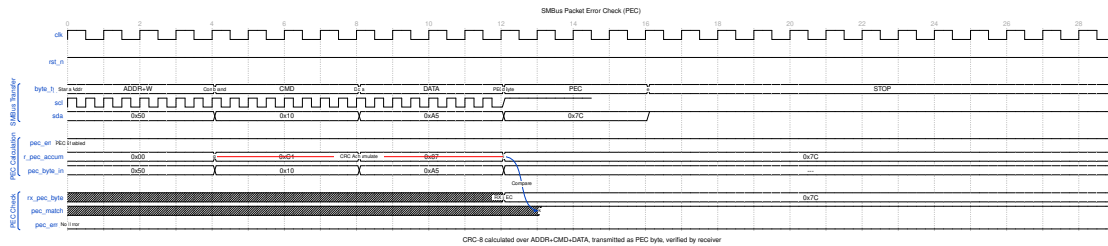


SMBus Arbitration

Both masters monitor SDA while transmitting. If a master drives 1 but reads 0 (wired-AND bus), it loses arbitration and backs off. The winner continues the transaction.

2.5.5 Waveform 1.5: Packet Error Check (PEC)

CRC-8 error detection for data integrity.



SMBus PEC

PEC is calculated over address, command, and data bytes using CRC-8. The PEC byte is transmitted after data and verified by the receiver to detect transmission errors.

2.6 Register Summary

Offset	Name	Access	Description
0x00	SMBUS_CO NTROL	RW	Global control (enable, mode, PEC, resets)
0x04	SMBUS_STA TUS	RO	Status flags and FSM state
0x08	SMBUS_CO MMAND	RW	Transaction type, command byte, start/stop
0x0C	SMBUS_SLA VE_ADDR	RW	Target slave address
0x10	SMBUS_DA TA	RW	Single data byte
0x14	SMBUS_TX_ FIFO	WO	Transmit FIFO write port
0x18	SMBUS_RX_ FIFO	RO	Receive FIFO read port
0x1C	SMBUS_FIF O_STATUS	RO	TX/RX FIFO levels and flags
0x20	SMBUS_CL K_DIV	RW	SCL clock divider
0x24	SMBUS_TIM EOUT	RW	Timeout threshold

Offset	Name	Access	Description
0x28	SMBUS_OWN_ADDR	RW	Own slave address (slave mode)
0x2C	SMBUS_INT_ENABLE	RW	Interrupt enable mask
0x30	SMBUS_INT_STATUS	W1C	Interrupt status
0x34	SMBUS_PEC	RW	PEC value (CRC-8)
0x38	SMBUS_BLOCK_COUNT	RW	Block transfer byte count

See [Chapter 5: Register Map](#) for full field definitions, reset values, and implementation limitations.

Next: [02_architecture.md](#)

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

3 APB SMBus - Register Map

The register map below is generated from `rtl/smbus/peakrdl/smbus_regs.rdl` and matches the decode in `rtl/smbus/smbus_regs.sv`. The register block decodes 6 address bits, so only offsets 0x00-0x3F are valid; higher offsets alias back onto this range. Undefined offsets read as 0, drop writes, and do not assert PSLVERR.

Access legend: RW = read/write, RO = read-only, WO = write-only, W1C = write-1-to-clear, AC = self-clearing (hardware clears the bit after the action completes).

3.1 Register Summary

Offset	Name	Access	Reset	Description
0x00	SMBUS_CONTROL	RW	0x00000000 0	Global control (enable, mode, PEC, resets)

Offset	Name	Access	Reset	Description
0x04	SMBUS_S TATUS	RO	0x00000000 0	Status flags and FSM state
0x08	SMBUS_C OMMAN D	RW	0x00000000 0	Transaction type, command byte, start/stop
0x0C	SMBUS_S LAVE_AD DR	RW	0x00000000 0	Target slave address (master mode)
0x10	SMBUS_D ATA	RW	0x00000000 0	Single data byte
0x14	SMBUS_T X_FIFO	WO	0x00000000 0	Transmit FIFO write port
0x18	SMBUS_R X_FIFO	RO	-	Receive FIFO read port (read pops)
0x1C	SMBUS_F IFO_STAT US	RO	0x00000000 0	TX/RX FIFO levels and flags
0x20	SMBUS_C LK_DIV	RW	0x0000000F 9	SCL clock divider
0x24	SMBUS_T IMEOUT	RW	0x002625A 0	Timeout threshold (clocks)
0x28	SMBUS_O WN_ADD R	RW	0x00000000 0	Own slave address (slave mode)
0x2C	SMBUS_I NT_ENAB LE	RW	0x00000000 0	Interrupt enable mask
0x30	SMBUS_I NT_STAT US	W1C	0x00000000 0	Interrupt status (write 1 to clear)
0x34	SMBUS_P EC	RW	0x00000000 0	PEC value (CRC-8)
0x38	SMBUS_B LOCK_CO	RW	0x00000000 0	Block transfer byte count

Offset	Name	Access	Reset	Description
	UNT			

3.2 SMBUS_CONTROL (0x00)

Bit	Name	Access	Reset	Description
0	master_en	RW	0	Enable master mode
1	slave_en	RW	0	Enable slave mode
2	pec_en	RW	0	Enable Packet Error Checking
3	fast_mode	RW	0	Clock speed select: 0=standard, 1=fast
4	fifo_reset	RW/AC	0	Reset TX/RX FIFOs (write 1, auto-clears)
5	soft_reset	RW/AC	0	Soft reset controller (write 1, auto-clears)
31:6	Reserved	RO	0	Reads 0

Note: `soft_reset` and `fast_mode` are accepted by the register block but are not currently consumed by `smbus_core` (soft reset resets nothing; SCL frequency depends solely on `clk_div`). See Implementation Limitations below.

3.3 SMBUS_STATUS (0x04)

Read-only. Software writes to this register are dropped (no PSLVERR). The sticky, clearable interrupt flags live in `SMBUS_INT_STATUS` (0x30), not here.

Bit	Name	Access	Description
0	busy	RO	Transaction in progress
1	bus_error	RO	Bus error detected (e.g. NAK)
2	timeout_error	RO	Transaction timeout (see limitation below)
3	pec_error	RO	PEC mismatch (see limitation below)
4	arb_lost	RO	Multi-master arbitration lost

Bit	Name	Access	Description
			(see limitation below)
5	nak_received	RO	NAK received from slave
6	slave_address	RO	Addressed as slave (see limitation below)
7	complete	RO	Transaction completed
11:8	fsm_state	RO	Current FSM state (debug)
31:12	Reserved	RO	Reads 0

3.4 SMBUS_COMMAND (0x08)

Bit	Name	Access	Reset	Description
3:0	trans_type	RW	0	Transaction type (see Transaction Types)
7:4	Reserved	RO	0	Reads 0
15:8	cmd_code	RW	0	SMBus command byte
16	start	RW/AC	0	Start transaction (write 1, auto-clears)
17	stop	RW/AC	0	Force stop/abort transaction (write 1, auto-clears)
31:18	Reserved	RO	0	Reads 0

3.4.1 Transaction Types

trans_type is a 4-bit field. Block Write and Block Read are separate encodings.

trans_type	Description
0	Quick Command
1	Send Byte
2	Receive Byte
3	Write Byte
4	Read Byte
5	Write Word
6	Read Word

trans_type	Description
7	Block Write
8	Block Read
9	Block Write-Block Read Process Call

3.5 SMBUS_SLAVE_ADDR (0x0C)

Bit	Name	Access	Reset	Description
6:0	slave_addr	RW	0	7-bit target slave address
31:7	Reserved	RO	0	Reads 0

The transfer direction (R/W) is derived from trans_type, not from an address bit. There is no writable R/W bit at bit 7; writes to bit 7 are ignored and it reads back 0.

3.6 SMBUS_DATA (0x10)

Bit	Name	Access	Reset	Description
7:0	data	RW	0	Single data byte for Send/Receive/Write/Read Byte-Word
31:8	Reserved	RO	0	Reads 0

3.7 SMBUS_TX_FIFO (0x14)

Write-only port into the 32-byte transmit FIFO. Each write pushes one byte. Reads return 0.

Bit	Name	Access	Reset	Description
7:0	tx_data	WO	0	Byte pushed into the TX FIFO
31:8	Reserved	RO	0	Reads 0

3.8 SMBUS_RX_FIFO (0x18)

Read-only port from the 32-byte receive FIFO. Each read pops one byte.

Bit	Name	Access	Description
7:0	rx_data	RO	Byte popped from the RX FIFO
31:8	Reserved	RO	Reads 0

3.9 SMBUS_FIFO_STATUS (0x1C)

Read-only FIFO level and flag register.

Bit	Name	Access	Description
5:0	tx_level	RO	TX FIFO occupancy (0-32)
6	tx_full	RO	TX FIFO full
7	tx_empty	RO	TX FIFO empty
13:8	rx_level	RO	RX FIFO occupancy (0-32)
14	rx_full	RO	RX FIFO full
15	rx_empty	RO	RX FIFO empty
31:16	Reserved	RO	Reads 0

3.10 SMBUS_CLK_DIV (0x20)

Bit	Name	Access	Reset	Description
15:0	clk_div	RW	0x00F9 (249)	SCL clock divider
31:16	Reserved	RO	0	Reads 0

SCL toggles every $\text{clk_div} + 1$ system-clock cycles, so $f_{\text{SCL}} = f_{\text{clk}} / (2 * (\text{clk_div} + 1))$. The reset value of 249 yields 100 kHz at $f_{\text{clk}} = 50$ MHz (200 kHz at $f_{\text{clk}} = 100$ MHz).

3.11 SMBUS_TIMEOUT (0x24)

Bit	Name	Access	Reset	Description
23:0	timeout	RW	0x2625A0 (2,500,000)	Timeout threshold in system-clock cycles
31:24	Reserved	RO	0	Reads 0

The reset value corresponds to ~25 ms at $f_{\text{clk}} = 100$ MHz. See Implementation Limitations below: the timeout counter is not currently enabled in `smbus_core`, so this threshold has no effect and `SMBUS_STATUS.timeout_error` never sets.

3.12 SMBUS_OWN_ADDR (0x28)

Bit	Name	Access	Reset	Description
6:0	own_addr	RW	0	7-bit own slave address
7	addr_en	RW	0	Enable own-address matching (slave mode)
31:8	Reserved	RO	0	Reads 0

3.13 SMBUS_INT_ENABLE (0x2C)

Bit	Name	Access	Reset	Description
0	complete_en	RW	0	Interrupt on transaction complete

Bit	Name	Access	Reset	Description
1	error_en	RW	0	Interrupt on bus error
2	tx_thresh_en	RW	0	Interrupt when TX FIFO below threshold
3	rx_thresh_en	RW	0	Interrupt when RX FIFO above threshold
4	slave_addr_en	RW	0	Interrupt when addressed as slave
31:5	Reserved	RO	0	Reads 0

3.14 SMBUS_INT_STATUS (0x30)

Write 1 to a bit to clear it.

Bit	Name	Access	Reset	Description
0	complete_int	W1C	0	Transaction completed
1	error_int	W1C	0	Bus error occurred
2	tx_thresh_int	W1C	0	TX FIFO below threshold
3	rx_thresh_int	W1C	0	RX FIFO above threshold
4	slave_addr_int	W1C	0	Device addressed

Bit	Name	Access	Reset	Description
31:5	Reserved	RO	0	as slave Reads 0

3.15 SMBUS_PEC (0x34)

Bit	Name	Access	Reset	Description
7:0	pec	RW	0	Current/expected PEC value (CRC-8, polynomial 0x07)
31:8	Reserved	RO	0	Reads 0

3.16 SMBUS_BLOCK_COUNT (0x38)

Bit	Name	Access	Reset	Description
5:0	block_count	RW	0	Byte count for block transfers (1-32)
31:6	Reserved	RO	0	Reads 0

Note: block_count semantics differ between reads and writes in the current RTL. On writes the first byte comes from SMBUS_DATA and block_count counts the additional FIFO bytes; on reads it counts the total byte count. The SMBus count byte is not placed on / consumed from the wire by the current logic.

3.17 Implementation Limitations

The register interface above matches the RTL, but several protocol features implied by these registers are not fully realized in the current smbus_core logic. These are noted so the register map does not promise behavior the hardware does not yet deliver:

- **Timeout detection is inactive.** The timeout counter enable is never driven, so SMBUS_TIMEOUT has no effect and SMBUS_STATUS.timeout_error cannot set.

- **SCL is not toggled during a transfer and is driven push-pull.** The generated SCL clock is not connected to the pin during bit phases, and the master does not use open-drain signaling, so clock stretching and multi-master arbitration are not supported.
- **PEC is not functional.** PEC generation does not accumulate valid transmitted data and received PEC is never compared, so `SMBUS_STATUS.pec_error` cannot set.
- **Hardwired status bits.** `arb_lost`, `pec_error`, and `slave_addressed` in `SMBUS_STATUS` are tied to 0. Slave mode (`SMBUS_OWN_ADDR`, `slave_addr_int`) is a stub and does not ACK, receive, or transmit.
- **Dead control bits.** `SMBUS_CONTROL.soft_reset` and `SMBUS_CONTROL.fast_mode` are accepted but not acted upon by the core.

For the authoritative, tracked status of these items see `rtl/smbus/IMPLEMENTATION_STATUS.md` and `rtl/smbus/TODO.md`.

Back to: [SMBus Specification Index](#)